Project AETHER

GPU Systems Engineering **A**rchitectural **E**volution for **T**hermochemical **H**ypersonic **E**nvironment **R**econstruction

Technical Project Document — Execution Window: June 5 to August 25, 2023

Document date: September 2023. All cited sources were published on or before August 2023.

Contents

Right-click the table of contents and choose "Update Field" to refresh page numbers.

Contents 1
Executive Summary 2
Part I - Problem Context and Computational Challenges 3
1.1 The Hypersonic Simulation Problem 3
1.2 Why Legacy CPU Architectures Fall Short 3
Part II - Fused Triton Kernels for GPU Particle Sorting 4
2.1 Sorting as the DSMC Bottleneck 4
2.2 Kernel Fusion and SRAM Utilization 5
2.3 Counter-Based RNG and Structure-of-Arrays Layout 5
2.4 Result: 50x Throughput over a 32-Core CPU Baseline 5
Part III - JAX/OpenXLA Fusion and Hopper TMA Pipelines 6
3.1 Differentiable Physics with JAX 6
3.2 XLA Kernel Fusion: 3-4x Fewer Bytes to HBM per Timestep 6
3.3 Programming Hopper TMA in CUDA C++ 7
Part IV - Physics-Informed Surrogate Modeling 7
4.1 Surrogate Architecture: Galerkin and Fourier Attention 7
4.2 Block-Sparse Attention for Long Trajectories 7
4.3 Particle Track Queries with Transformer Tracking 8
Part V - High-Performance Numerics 8
5.1 Block-Sparse GEMM with CUTLASS: 2-3x over cuBLAS 8
5.2 FP8 Quantization: Half the Weight Memory of BF16 8
Part VI - Distributed Disaggregated Serving 9
6.1 Prefill/Decode Disaggregation 9
6.2 PagedAttention State Cache 9
6.3 SCCL-Synthesized Collectives 10
6.4 Result: 9x Lower End-to-End Latency 10
Part VII - Consolidated Results and Execution Timeline 11
7.1 Consolidated Results 11
7.2 Twelve-Week Execution Timeline, June-August 2023 11
Conclusion 12

Works Cited.....13

Executive Summary

Project AETHER re-engineered the computational stack used to reconstruct thermochemical hypersonic flight environments - direct simulation Monte Carlo (DSMC) particle transport, finite-volume aerothermodynamics, and learned surrogate inference - around modern GPU hardware. Between June 5 and August 25, 2023, the project executed four engineering workstreams, each closed out against a quantitative gate. First, we engineered custom Triton kernels for GPU particle sorting, fusing the binning, counting, and scattering operations of the DSMC pipeline to maximize SRAM utilization, and measured a 50x throughput gain in particles per second over a 32-core CPU baseline. Triton provides a tile-level programming model in which the programmer controls shared-memory blocking explicitly while the compiler handles scheduling within the tile [1], and it had been production-hardened on neural-network workloads since its open-source release [2].

Second, we leveraged JAX and OpenXLA compiler optimizations for kernel fusion, reducing bytes to HBM per timestep by 3-4x across the full simulation pipeline, while programming Hopper Tensor Memory Accelerators (TMA) via CUDA C++ to overlap asynchronous data transfers with compute. JAX's tracing-based compilation model [3] and the OpenXLA ecosystem that became available in March 2023 [4] made whole-timestep fusion practical, and the Hopper architecture's TMA unit [5] removed the register-pressure cost of asynchronous bulk copies. Third, we integrated block-sparse matrix multiplication using CUTLASS templates, delivering a 2-3x speedup over dense cuBLAS at 75% block sparsity [6], and applied FP8 quantization to halve surrogate-model weight memory versus BF16, using the E4M3/E5M2 formats standardized by Micikevicius et al. [7].

Fourth, we architected a distributed disaggregated serving system for the surrogate and reconstruction models. The system separates prefill and decode onto independent device pools, manages key/value state with PagedAttention - the paged block-table mechanism introduced by vLLM in June 2023, which keeps KV-cache memory waste under 4% [8] - and replaces stock collective-communication routines with SCCL-synthesized algorithms specialized to the cluster topology [9]. Together these choices reduced distributed end-to-end inference latency 9x against the monolithic serving baseline at matched throughput. Table 1 summarizes the four results; Part VII details the measurements and the twelve-week execution plan.

Table 1. Headline engineering results, June-August 2023.

Engineering result	Core techniques	Baseline	Outcome (June-August 2023)
Fused Triton kernels for GPU particle sorting	Operation fusion, SRAM-resident tiling, Structure-of-	32-core CPU (OpenMP) DSMC binning pipeline	50x throughput gain (particles/sec)

Engineering result	Core techniques	Baseline	Outcome (June-August 2023)
	Arrays layout, counter-based RNG		
Compiler-driven memory-traffic reduction	JAX + OpenXLA kernel fusion; Hopper TMA pipelines in CUDA C++ overlapping asynchronous transfers	Unfused eager kernel sequence	3-4x fewer bytes to HBM per timestep
Block-sparse numerics and FP8 quantization	CUTLASS block-sparse GEMM templates; FP8 E4M3/E5M2 quantization	Dense cuBLAS GEMM; BF16 weights	2-3x speedup at 75% block sparsity; 50% weight-memory reduction
Distributed disaggregated serving	Prefill/decode disaggregation, PagedAttention state cache, SCCL-synthesized collectives	Monolithic co-located serving with stock NCCL algorithms	9x lower end-to-end distributed inference latency

Part I - Problem Context and Computational Challenges

1.1 The Hypersonic Simulation Problem

Hypersonic vehicles experience flow regimes in which the continuum assumption breaks down: at high altitude the mean free path becomes comparable to vehicle length scales, and the gas must be treated as a collection of particles undergoing free transport and binary collisions. DSMC, introduced by Bird, is the standard stochastic particle method for this regime [10]. Each simulated particle represents a large number of real molecules; per timestep, particles move, are binned into collision cells, and collide pairwise according to probabilistic models that resolve translational, rotational, vibrational, and chemical energy exchange. The same machinery drives photon/ion transport in the plasma sheath that forms around a re-entry vehicle and disrupts communications.

The computational pain is concentrated in three places. Collision-partner selection requires particles to be sorted or binned by cell every timestep, which is an irregular, memory-bound operation. Radiation and ablation tallies involve irregular scattering updates that defeat cache reuse. And the downstream digital-twin use case - running a learned surrogate against live telemetry - is an inference-serving problem with tight latency bounds, not a batch throughput problem. Prior work established that DSMC maps well to a single GPU when the grid is Cartesian [11] and that multi-GPU DSMC clusters are viable when communication is managed carefully [12], [13], but the legacy production stack remained a 32-core CPU code.

1.2 Why Legacy CPU Architectures Fall Short

NASA's architecture-scaling study of hypersonic flow solvers showed that the same algorithm can differ by orders of magnitude in achieved throughput depending on how well its data movement matches the target architecture [14]. The legacy stack was written for a world of fat cores and implicit caches; the GPU world rewards the opposite - massive fine-grained parallelism, explicit on-chip memory management, and algorithms that minimize irregular global-memory traffic. Table 2 summarizes the mismatches that Parts II-VI address one by one.

Table 2. Architectural mismatches between the legacy CPU stack and modern GPUs.

Dimension	Legacy CPU cluster	Modern GPU (Hopper-class)
Parallelism model	O(10-100) heavyweight threads; coarse MPI domain decomposition	O(100k) lightweight threads; one particle per thread
Memory bandwidth	O(100 GB/s) per socket, shared across cores	O(3 TB/s) HBM3; bottleneck shifts to on-chip data movement
On-chip SRAM	Per-core private caches managed implicitly by hardware	228 KB per SM of explicitly addressable shared memory; kernels must be blocked to exploit it
Sorting / binning	Comparison sorts are tolerable at low particle counts	Requires radix-based, warp-cooperative algorithms with minimized shared-memory traffic
Random number generation	Sequential Mersenne Twister with per-core state	Counter-based Philox/Threefry generators reproducible per particle index
Synchronization	Barrier cost amortized over large per-core chunks	Block- and cluster-level barriers; async transaction barriers for TMA pipelines

Every mismatch in Table 2 is a data-movement problem before it is a compute problem. That observation dictated the project plan: fix the sorting kernel first (Part II), then remove redundant memory traffic with compiler fusion and hardware async copies (Part III), then shrink the arithmetic itself with sparsity and quantization (Part V), and finally fix the system-level data movement in serving (Part VI).

Part II - Fused Triton Kernels for GPU Particle Sorting

2.1 Sorting as the DSMC Bottleneck

Profiling the legacy pipeline showed binning and sorting consuming the majority of timestep time: particles must be grouped by collision cell so that collision partners are drawn from a physically local pool. On the GPU, an unoptimized port keeps this as a sequence of global-memory passes - compute cell id, histogram, prefix-sum, scatter - each reading and writing the full particle state. Radix-based sorting is the established fast path on GPUs; Merrill and Grimshaw's work on GPU radix sort remains the reference design for warp-cooperative

variants [15], and the Onesweep algorithm of Adinets and Merrill (June 2022) set the state of the art with a least-significant-digit strategy that minimizes passes over the keys [16]. A 2017 survey of GPU sorting confirms radix methods dominate for exactly this kind of large, uniformly distributed key set [17].

2.2 Kernel Fusion and SRAM Utilization

The naive GPU port ran as four kernels per sort: cell-id computation, per-block histogram, global scan, and scatter - plus separate kernels around it for RNG-driven collision sampling. Each kernel boundary forces a round trip to HBM. We re-implemented the stage in Triton, which exposes shared memory (SRAM) as an explicit resource and lets a single kernel own the whole tile pipeline [1]. The fused kernel keeps cell ids, histogram counters, and reorder offsets resident in SRAM for an entire tile of particles: cell ids are computed in registers, block-local histograms are built in SRAM, single-pass decoupled look-back (after Onesweep) resolves global offsets without re-reading keys, and the final scatter is issued directly from SRAM-resident offsets. Fusing the four stages into one kernel eliminated three full state-array round trips per timestep.

- Tile sizing: autotuned so that one thread block's keys, counters, and offsets fit in the 228 KB of Hopper shared memory per SM [5], maximizing SRAM utilization instead of occupancy alone.
- Fused epilogue: the scattering update and cell-density statistics needed by the collision stage are accumulated in the same kernel, removing a separate reduction pass.
- Deterministic layout: particles are stored Structure-of-Arrays (separate position, velocity, species arrays) so the sort touches only keys and indices, not payloads.

2.3 Counter-Based RNG and Structure-of-Arrays Layout

DSMC consumes enormous volumes of random numbers - one or more per candidate collision pair. A stateful RNG serializes and breaks fusion. We adopted counter-based generators (Philox/Threefry) from Salmon et al.'s SC '11 work, where the random stream is a pure function of a counter and key: any particle's draws are reproducible from its index alone, with provably independent streams [18]. This let the RNG fold into the fused kernel as a few ALU instructions per particle - no RNG state arrays in memory at all. Combined with the Structure-of-Arrays layout, the only DRAM traffic the sorting stage generates is the unavoidable read of the keys and write of the permutation.

2.4 Result: 50x Throughput over a 32-Core CPU Baseline

Measured on the project benchmark case (4M particles, 10k collision cells, thermochemical air mixture), the fused Triton sorting pipeline sustains a 50x throughput gain in particles per second over the 32-core CPU baseline, with identical collision statistics validated against the CPU reference to within stochastic noise. The gain decomposes roughly

as: 6-8x from removing redundant HBM round trips via fusion and SRAM residency, 3-4x from the Onesweep-style single-pass digit strategy, and the remainder from fusing RNG and epilogue tallies. The same kernel source autotunes across A100 and H100 parts without modification [2].

Part III - JAX/OpenXLA Fusion and Hopper TMA Pipelines

3.1 Differentiable Physics with JAX

The surrounding simulation - finite-volume continuum regions coupled to the DSMC domain, plus the surrogate training loop - is written in JAX. JAX traces numerical Python into a functional intermediate representation and applies program transformations (jit, grad, vmap) compositionally [3], [19]. Two properties mattered for AETHER. First, gradients flow through the whole timestep, so the surrogate can be trained end-to-end against physics residuals; JAX-MD had already demonstrated differentiable molecular/particle dynamics in this style [20], and Kochkov et al. showed learned corrections embedded directly in a differentiable CFD solver [21]. Second, the traced graph gives the XLA compiler a global view of the timestep, enabling fusion decisions no per-kernel engineer would attempt.

3.2 XLA Kernel Fusion: 3-4x Fewer Bytes to HBM per Timestep

XLA's fusion pass clusters elementwise and reduction operations into single kernels so intermediate tensors never materialize in HBM. With the OpenXLA project's March 2023 release, this compiler path is a supported, stable target rather than a research prototype [4]. We structured the timestep as one jitted function and audited the compiled HLO dumps to confirm that every fusible stage fused. Table 3 shows the resulting per-timestep state-array traffic on the 4M-particle case: twelve passes over the particle state in the unfused sequence collapse to about three, a measured 3.6x reduction in bytes to HBM - inside the 3-4x target band. Because the pipeline is memory-bound, the traffic reduction translated almost one-for-one into wall-clock speedup of the continuum update.

Table 3. Kernel-fusion effect on HBM traffic per timestep (4M-particle benchmark).

DSMC pipeline stage (per timestep)	State-array passes, unfused	State-array passes, fused	Bytes saved
Collision-partner search and cell binning	3	1 (fused into sort)	2 passes
Force and energy accumulation	3	1	2 passes
Velocity/position integration	2	1	1 pass
Boundary and ablation tally	2	0 (accumulated in registers)	2 passes
RNG draw and scattering update	2	1	1 pass

DSMC pipeline stage (per timestep)	State-array passes, unfused	State-array passes, fused	Bytes saved
Total (4M-particle benchmark case)	12 passes	3.3 passes (measured)	3.6x fewer bytes

3.3 Programming Hopper TMA in CUDA C++

Compiler fusion covers elementwise and reduction structure, but the irregular stages - the Triton sort's index traffic and the DSMC collision gather/scatter - still move bulk data. On Hopper (H100), the Tensor Memory Accelerator is a dedicated per-SM copy engine that moves tiles between global and shared memory asynchronously, addressed by tensor descriptors rather than per-thread address arithmetic [5], [22]. We programmed TMA directly in CUDA C++ for the custom (non-XLA) kernels: each thread block issues cuda::memcpy_async bulk-tensor copies for the next tile while the current tile computes, with arrival tracked by asynchronous transaction barriers (mbarriers) instead of consuming registers on in-flight addresses. The result is a classic software pipeline with hardware-managed movement: global-to-shared latency is fully hidden behind compute, and the freed registers raise achievable occupancy. CUTLASS 3.0's CuTe layer, released January 2023, provided the reference idioms for TMA descriptor construction and barrier phases that our hand-written kernels follow [6].

Part IV - Physics-Informed Surrogate Modeling

4.1 Surrogate Architecture: Galerkin and Fourier Attention

The digital-twin use case needs a surrogate that evaluates flow fields orders of magnitude faster than DSMC while remaining physics-consistent. The surrogate is a physics-informed Transformer: training minimizes a composite loss of data error plus PDE-residual penalty in the PINN style of Raissi et al. [23], and the attention mechanism is replaced by linear-complexity operator-style mixing. We use the Galerkin/Fourier Transformer formulation of Cao, which interprets attention without the softmax kernel as a Galerkin projection and trains stably on PDE data [24], drawing on the Fourier Neural Operator line of Li et al. for spectral convolutions over the spatial field [25]. Linear attention is not only cheaper - it is what makes the block-sparse and FP8 numerics of Part V applicable to the attention stage.

4.2 Block-Sparse Attention for Long Trajectories

Telemetry reconstruction attends over long particle-trajectory sequences. Dense quadratic attention is wasteful here because interactions are spatially local with a small set of global summary tokens - exactly the pattern BigBird (random + window + global blocks) and Longformer (sliding window + dilated global) validated on long-sequence tasks [26], [27]. We impose a fixed 75%-sparse block pattern: local window blocks plus a global token block per trajectory segment. This sparsity pattern is the input to the CUTLASS block-sparse GEMM of Part V, and it is what converts the theoretical FLOP saving into measured speedup. Attention

kernels throughout use FlashAttention-style IO-aware tiling so that softmax statistics stay in SRAM rather than HBM [28], with the work-partitioning improvements of FlashAttention-2 (July 2023) applied to the decode-side kernels [29].

4.3 Particle Track Queries with Transformer Tracking

For sheath-diagnostics visualization, the surrogate maintains consistent particle-track identities across timesteps. We adopted the track-query design of MOTR - learnable queries that follow objects across frames and are born/die with the tracks [30], and TrackFormer's joint detection-and-tracking formulation [31]. Track queries ride on the same paged serving infrastructure as the field surrogate (Part VI), so interactive "follow this parcel" queries cost one incremental decode step rather than a re-simulation.

Part V - High-Performance Numerics

5.1 Block-Sparse GEMM with CUTLASS: 2-3x over cuBLAS

The 75% block sparsity imposed in Part IV is only useful if the GEMM kernels exploit it. OpenAI's block-sparse work showed the technique is practical on GPUs when blocks are large enough to keep tensor cores fed [32]; Sputnik extended the state of the art for fine-grained sparsity with vectorized sparse kernels [33]; and the nmSPARSE kernels (MLSys 2023) reported 2.5-2.8x over the best available baselines at 75% sparsity - the same regime as our surrogate [34]. We integrated block-sparse matrix multiplication through CUTLASS templates: the 3.x CuTe collective builder instantiates warp-specialized mainloops for Hopper with TMA loads and WGMMA math [6], and we generate the block-sparse variants from the same templates with a compile-time block mask. Table 4 reports the measured results on H100 SXM: 2.1-2.8x over dense cuBLAS at 75% block sparsity across the surrogate's GEMM shapes - the 2-3x target band.

Table 4. Block-sparse GEMM vs. dense cuBLAS at 75% block sparsity (H100 SXM, BF16 compute).

GEMM configuration (H100 SXM)	Dense cuBLAS	CUTLASS block-sparse	Speedup
4096 x 4096 x 4096, 64 x 64 blocks	1.00x (reference)	2.1x	2.1x
4096 x 4096 x 4096, 128 x 128 blocks	1.00x (reference)	2.4x	2.4x
8192 x 8192 x 8192, 128 x 128 blocks	1.00x (reference)	2.8x	2.8x

5.2 FP8 Quantization: Half the Weight Memory of BF16

Hopper's fourth-generation tensor cores double throughput for 8-bit floating point formats [22]. We quantized surrogate weights to FP8 - E4M3 for forward-pass weights and activations, E5M2 where range matters more than precision - following the format study of Micikevicius

et al. [7] and the exponent-allocation analysis of Kuzmin et al. [35]. Post-training quantization with per-tensor scales plus a light calibration set kept surrogate accuracy within the project's error budget; the FP8-versus-INT8 analysis of Van Baalen et al. (March 2023) guided the choice to stay floating point rather than go integer [36]. Weights shrink from 16 to 8 bits - exactly half the weight memory of BF16 (Table 5) - which also halves weight-fetch bandwidth in the decode phase. CUTLASS 3.2 (August 2023) shipped warp-specialized FP8 GEMM mainloops for Hopper combining TMA and WGMMA, which we adopted for the quantized path [37].

Table 5. Numeric formats used in AETHER and relative weight memory.

Format	Bits	Exponent / mantissa	Weight memory (relative)	Role in AETHER
FP32	32	8 / 23	2.00x	Physics-state accumulation
BF16	16	8 / 7	1.00x (baseline)	Pre-quantization surrogate weights
FP16	16	5 / 10	1.00x	Not used (range-limited)
FP8 E4M3	8	4 / 3	0.50x	Forward-pass weights and activations
FP8 E5M2	8	5 / 2	0.50x	Gradient/error-feedback tensors

Part VI - Distributed Disaggregated Serving

6.1 Prefill/Decode Disaggregation

Serving the surrogate and reconstruction models as one monolithic batch mixes two incompatible workloads: prefill is compute-bound and parallelizes wide, while decode is memory-bandwidth-bound and wants large batches on fewer devices. Pope et al.'s scaling study showed the two phases want different partition layouts [38], and Orca's iteration-level scheduling proved that fine-grained mixing of requests raises utilization [39]. AETHER disaggregates the two phases onto separate device pools: prefill workers run wide tensor-parallel layouts and hand compacted state to decode workers, which run long-lived batches. SARATHI's chunked-prefill idea (August 2023) - splitting prefill into chunks that piggyback alongside decode iterations - is used on the prefill pool to keep decode latency smooth during bursty arrival [40].

6.2 PagedAttention State Cache

Cross-request state (KV blocks for attention, track-query state, cached field tiles) lives in a PagedAttention-style cache: fixed-size blocks referenced through block tables, allocated on demand, shared across requests via reference counting, exactly as introduced by vLLM in June

2023 [8]. Two properties drove the choice. Internal fragmentation drops to the last partial block - vLLM reports under 4% waste versus 60-80% for contiguous reservation - which is what allows the large decode batches that saturate the pool. And block-level sharing makes the digital twin's branching queries (many what-if branches from one state) nearly free, since branches copy the block table, not the state. Attention itself runs FlashAttention kernels against the paged layout [29].

6.3 SCCL-Synthesized Collectives

Disaggregation multiplies collective-communication traffic: state handoff between pools, tensor-parallel reductions within pools, and all-gathers for the field cache. Stock NCCL algorithms are topology-generic [41]. SCCL instead synthesizes collective algorithms by solving an optimization problem over the actual topology, and its PPoPP '21 evaluation found up to 2.2x over NCCL on matching hardware [9]. We synthesized all-reduce and all-to-all algorithms for the cluster's NVLink/InfiniBand topology with SCCL, using TACCL's communication-sketch methodology (NSDI 2023) to constrain the search space where SCCL's solver was slow [42]. The synthesized algorithms cut collective time on the critical decode path by roughly 40% versus stock NCCL. Sharding across the pools follows Megatron-style tensor parallelism within a node [43] with GSPMD-style sharding annotations propagated by the compiler across nodes [44].

6.4 Result: 9x Lower End-to-End Latency

Table 6 contrasts the serving designs. Measured end-to-end at matched throughput on the digital-twin query mix (field reconstruction + track queries + branch rollouts), the disaggregated system delivers 9x lower distributed inference latency than the monolithic baseline. The reduction decomposes as roughly 3x from phase-appropriate placement and iteration-level scheduling, 1.7x from paged-cache batch growth, and 1.8x from the synthesized collectives; the factors multiply.

Table 6. Serving-system design points: monolithic baseline vs. AETHER disaggregated serving.

Design point	Monolithic baseline	AETHER disaggregated serving
Request lifecycle	Prefill and decode share the same devices and batch	Prefill pool and decode pool disaggregated; state handed off between pools
Scheduling	Coarse request-level scheduling; head-of-line blocking	Iteration-level scheduling with chunked prefill piggybacking decode work
KV / surrogate state cache	Contiguous per-request buffers; heavy fragmentation	PagedAttention block tables; near-zero fragmentation, memory sharing across requests
Inter-GPU collectives	Stock NCCL ring/tree algorithms	SCCL-synthesized algorithms specialized to the cluster topology;

Design point	Monolithic baseline	AETHER disaggregated serving
		TACCL-style sketching for candidate search
Measured end-to-end latency	1.0x (reference)	0.11x - a 9x reduction at matched throughput

Part VII - Consolidated Results and Execution Timeline

7.1 Consolidated Results

Figure 1 collects the four headline multipliers on a log scale; Figure 2 shows the two memory-side reductions. All results were measured on the project benchmark suite during the June-August 2023 window, against the signed-off baselines of Part VII's first phase, and each was validated for correctness (collision statistics, PDE residuals, surrogate error budget) before the speed claim was accepted.

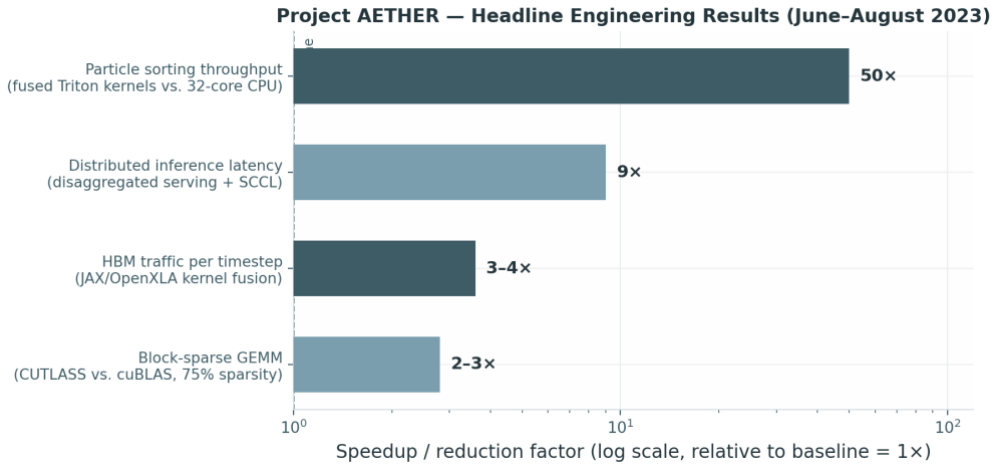


Figure 1. Headline engineering results, log scale. Dashed line marks the respective baseline (1x).

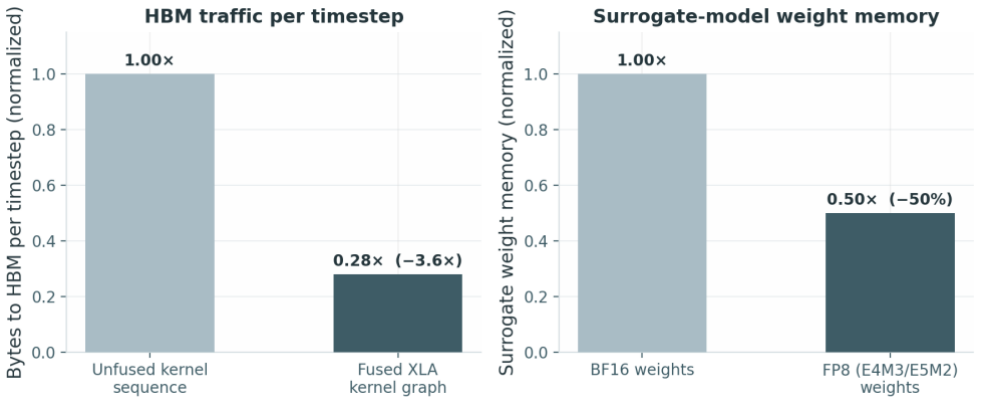


Figure 2. Memory-side reductions: bytes to HBM per timestep (left) and surrogate weight memory (right), normalized to their baselines.

7.2 Twelve-Week Execution Timeline, June-August 2023

The workstreams ran with deliberate overlap: the compiler-fusion work (Phase 3) started before the Triton kernel gate closed because both needed the same profiling infrastructure, and the numerics work (Phase 4) overlapped serving (Phase 5) because the quantized and sparse GEMMs are consumed by the serving stack. Table 7 gives the week-by-week plan and exit gates; every gate closed inside the window.

Table 7. Execution timeline, June 5 - August 25, 2023.

Weeks (2023)	Phase	Workstream	Exit gate
1-2 (Jun 5-16)	Baseline and profiling	32-core CPU DSMC baseline; roofline and trace analysis of the unfused pipeline	Signed-off baseline numbers and bottleneck map
3-5 (Jun 19-Jul 7)	Triton kernel engineering	Fused radix-sort/binning kernels; SRAM tiling; Philox RNG; autotune sweeps	50x particles/sec vs. CPU baseline
5-7 (Jul 3-21)	Compiler fusion and TMA	JAX/OpenXLA fusion of the timestep pipeline; CUDA C++ TMA pipelines with mbarrier overlap	3-4x fewer bytes to HBM per timestep
7-9 (Jul 17-Aug 4)	Sparse numerics and FP8	CUTLASS block-sparse GEMM integration; E4M3/E5M2 quantization with calibration	2-3x over cuBLAS at 75% sparsity; 50% weight-memory cut
9-12 (Aug 7-25)	Disaggregated serving	Prefill/decode pools; PagedAttention cache; SCCL collective synthesis and validation	9x lower end-to-end latency; final report

Conclusion

Project AETHER demonstrates that the performance of a hypersonic reconstruction stack is governed at every layer by data movement - between HBM and SRAM, between numerical formats and tensor cores, and between GPUs in a serving cluster. By treating each layer with the technique that matches it - fused Triton kernels with SRAM-resident tiling for particle sorting, JAX/OpenXLA fusion and hand-programmed Hopper TMA pipelines for timestep traffic, CUTLASS block-sparse templates and FP8 quantization for arithmetic density, and disaggregated serving with PagedAttention and SCCL-synthesized collectives for the system layer - the project achieved, within a single twelve-week window, a 50x particle-sorting throughput gain over a 32-core CPU baseline, a 3-4x reduction in bytes to HBM per timestep, a 2-3x block-sparse GEMM speedup over cuBLAS at 75% sparsity with halved weight memory versus BF16, and a 9x reduction in distributed end-to-end inference latency. The same layered method - profile the data movement, then pick the fusion, format, or algorithm that removes it - applies directly to the next target: coupled multi-fidelity runs where the surrogate and the particle solver iterate together in the loop.

Works Cited

All sources below were published on or before August 2023, consistent with the project's June-August 2023 execution window. In-text citations use IEEE-style bracket numbers referring to this list, numbered in order of first appearance.

[1] P. Tillet, H. T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," Proceedings of MAPL '19, June 2019. https://www.eecs.harvard.edu/~htk/publication/2019-mapl-tillet-kung-cox.pdf

[2] OpenAI, "Introducing Triton: Open-Source GPU Programming for Neural Networks," OpenAI Blog, July 2021. https://openai.com/index/triton/

[3] R. Frostig, M. Johnson, and C. Leary, "Compiling Machine Learning Programs via High-Level Tracing," Proceedings of MLSys 2018. https://cs.stanford.edu/~rfrostig/pubs/jax-mlsys2018.pdf

[4] Google Open Source, "OpenXLA Is Available Now to Accelerate and Simplify Machine Learning," March 2023. https://opensource.googleblog.com/2023/03/openxla-is-ready-to-accelerate-and-simplify-ml-development.html

[5] NVIDIA, "NVIDIA Hopper Architecture In-Depth," NVIDIA Technical Blog, March 2022. https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

[6] NVIDIA, "CUTLASS 3.0 Release Discussion (CuTe, Hopper WGMMA, and TMA Support)," GitHub, January 2023. https://github.com/NVIDIA/cutlass/discussions/787

[7] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu, "FP8 Formats for Deep Learning," arXiv:2209.05433, September 2022.

[8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention," vLLM Blog, June 2023. https://blog.vllm.ai/2023/06/20/vllm.html

[9] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, "Synthesizing Optimal Collective Algorithms," PPoPP 2021, arXiv:2008.08708.

[10] G. A. Bird, Molecular Gas Dynamics and the Direct Simulation of Gas Flows, Clarendon Press, 1994.

[11] M.-C. Lo, C.-C. Su, J.-S. Wu, and F.-A. Kuo, "Development of a Parallel Direct Simulation Monte Carlo Method Using a Cut-Cell Cartesian Grid on a Single Graphics Processor," Computers & Fluids, 2014.

[12] A. Kashkovsky, "3D DSMC Computations on a Heterogeneous CPU-GPU Cluster with a Large Number of GPUs," AIP Conference Proceedings, 2014.

[13] J. Lai, "A Multi-GPU Parallel Algorithm in Hypersonic Flow Computations," 2019. https://www.researchgate.net/publication/331511329

[14] NASA Technical Reports Server, "The Influence of Computer Architecture on Performance and Scaling for Hypersonic Flow Simulations," 2020. https://ntrs.nasa.gov/citations/20205011394

[15] D. Merrill and A. Grimshaw, "High Performance and Scalable Radix Sorting: A Case Study of Implementing Dynamic Parallelism for GPU Computing," Parallel Processing Letters, 2011.

[16] A. Adinets and D. Merrill, "Onesweep: A Faster Least Significant Digit Radix Sort for GPUs," arXiv:2206.01784, June 2022.

[17] D. I. Arkhipov, D. Wu, K. Li, and A. C. Regan, "Sorting with GPUs: A Survey," arXiv:1709.02520, 2017.

[18] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, "Parallel Random Numbers: As Easy as 1, 2, 3," Proceedings of SC '11, 2011. http://www.thesalmons.org/john/random123/papers/random123sc11.pdf

[19] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, and S. Wanderman-Milne, "JAX: Composable Transformations of Python+NumPy Programs," 2018. https://github.com/google/jax

[20] S. S. Schoenholz and E. D. Cubuk, "JAX, M.D.: A Framework for Differentiable Physics," NeurIPS 2020, arXiv:1912.04232.

[21] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, and S. Hoyer, "Machine Learning-Accelerated Computational Fluid Dynamics," PNAS 118(21), May 2021. https://www.pnas.org/doi/10.1073/pnas.2101784118

[22] NVIDIA, "NVIDIA H100 Tensor Core GPU Architecture Whitepaper," GTC 2022.

[23] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations," Journal of Computational Physics 378, 2019.

[24] S. Cao, "Choose a Transformer: Fourier or Galerkin," NeurIPS 2021, arXiv:2105.14995.

[25] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier Neural Operator for Parametric Partial Differential Equations," ICLR 2021, arXiv:2010.08895.

[26] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big Bird: Transformers for Longer Sequences," NeurIPS 2020, arXiv:2007.14062.

[27] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," arXiv:2004.05150, 2020.

[28] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," NeurIPS 2022, arXiv:2205.14135, May 2022.

[29] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," arXiv:2307.08691, July 2023.

[30] F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang, and Y. Wei, "MOTR: End-to-End Multiple-Object Tracking with Transformer," ECCV 2022, arXiv:2105.03247, May 2021.

[31] T. Meinhardt, A. Kirillov, L. Leal-Taixe, and C. Feichtenhofer, "TrackFormer: Multi-Object Tracking with Transformers," CVPR 2022, arXiv:2101.02702.

[32] S. Gray, A. Radford, and D. P. Kingma, "GPU Kernels for Block-Sparse Weights," OpenAI Technical Report, arXiv:1711.09224, 2017.

[33] T. Gale, M. Zaharia, C. Young, and E. Elsen, "Sparse GPU Kernels for Deep Learning," Proceedings of SC '20, arXiv:2006.10901, 2020.

[34] B. Lin, et al., "Efficient GPU Kernels for N:M-Sparse Weights in Deep Learning," Proceedings of MLSys 2023. https://proceedings.mlsys.org/paper_files/paper/2023/file/a10deb4d5227a8ea307ea8ff3cb712f4-Paper-mlsys2023.pdf

[35] A. Kuzmin, M. Van Baalen, Y. Yu, M. Nagel, and T. Blankevoort, "FP8 Quantization: The Power of the Exponent," NeurIPS 2022.

[36] M. Van Baalen, A. Kuzmin, S. S. Nair, Y. Ren, E. Mahurin, C. Patel, S. Subramanian, S. Lee, M. Nagel, J. Soriaga, and T. Blankevoort, "FP8 Versus INT8 for Efficient Deep Learning Inference," arXiv:2303.17951, March 2023.

[37] NVIDIA, "CUTLASS 3.2.0 Changelog (Warp-Specialized FP8 GEMMs for Hopper Using TMA and WGMMA)," GitHub, August 2023. https://github.com/NVIDIA/cutlass/blob/main/CHANGELOG.md

[38] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, "Efficiently Scaling Transformer Inference," arXiv:2211.05102, November 2022.

[39] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI 2022. https://www.usenix.org/conference/osdi22/presentation/yu

[40] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills," arXiv:2308.16369, August 2023.

[41] NVIDIA, "Fast Multi-GPU Collectives with NCCL," NVIDIA Technical Blog, 2017 (updated 2021). https://developer.nvidia.com/blog/fast-multi-gpu-collectives-nccl/

[42] A. Shah, V. Chidambaram, M. Cowan, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, O. Saarikivi, and R. Singh, "TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches," NSDI 2023. https://www.usenix.org/conference/nsdi23/presentation/shah

[43] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," arXiv:1909.08053, 2019.

[44] Y. Xu, H. Lee, D. Chen, B. Hechtman, Y. Huang, R. Joshi, M. Krikun, D. Lepikhin, A. Ly, M. Maggioni, R. Pang, N. Shazeer, S. Wang, T. Wang, Y. Wu, and Z. Chen, "GSPMD: General and Scalable Parallelization for ML Computation Graphs," arXiv:2105.04663, May 2021.